

Software Quality Assurance and Management

第4周：测试管理与敏捷测试实践

Week 4: Test Management & Agile Testing Practices

By: Fahad Sabah

计算机学院,

北京工业大学,中国

College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn



项目交付物/截止日期

Project Deliverables/Deadlines

LEBCS Project
Template Shared

Overleaf/Latex
Template for paper
writing

Type A (Portfolio Project) or the Type B (Survey/Review Paper):

- ❑ **Submission Deadline:** Thursday, June 25, 2026
- ❑ **Reviews & Plagiarism Report:** Friday, June 26, 2026
- ❑ **Final Submission Deadline:** Thursday, July 03, 2026

项目交付物/截止日期

Project Deliverables/Deadlines

Survey/Review Paper:

1. **Title & Authors:** concise, informative.
2. **Abstract:** 150–250 words — aim, methods, main results, key insights.
3. **Keywords:** 4–6 terms.
4. **Introduction:** context, motivation, research questions/objectives.
5. **Related Work / Background:** short review situating topic and bibliometric approach and existing review/survey papers.
6. **Data & Search Strategy:** data source(s) (e.g., Web of Science/Scopus), date range, search strings, inclusion/exclusion, record counts. (Include full search string in Appendix.)
7. **Preprocessing & Tools:** describe cleaning, deduplication, and software used (bibliometrix/BibShiny, VOSviewer) plus versions and key parameters.

项目交付物/截止日期

Project Deliverables/Deadlines

8. **Methods / Analysis Protocol:** outline metrics and analyses performed — descriptive stats (annual growth, sources, authors, countries), citation analysis, co-citation, bibliographic coupling, co-authorship, keyword co-occurrence, thematic mapping, trend analysis; network construction thresholds; visualization choices.
9. **Results:** structured by analysis type; include tables and readable figures (maps, networks, trend plots). Provide concise interpretation for each result.
10. **Discussion:** synthesize findings, compare with prior work, highlight thematic clusters and research gaps.
11. **Conclusions & Future Work:** main takeaways and suggested directions.
12. **Data & Code Availability / Reproducibility Statement:** link to scripts, datasets, and instructions for reproducing key figures.
13. **References**
14. **Appendices:** raw search strings, additional tables/figures, parameter settings, short README for code.

Week 4: Test Management & Agile Testing Practices 第4周：测试管理与敏捷测试实践

项目交付物/截止日期

Project Deliverables/Deadlines

- ☐ Search Strategy & Data (5 marks)
- ☐ Methodology & Analysis (5 marks)
- ☐ Use of Tools & Visualizations (5 marks)
- ☐ Results & Insights (5 marks)
- ☐ Writing, Structure & References (5 marks)
- ☐ Submission to the journal (5 marks)

简介

Introduction

1. To identify all defects existing in the software, remove them, and achieve error-free operation.
 2. Testing is vital for the success of a system.
 3. Inadequate testing or non-testing leads to errors which may not appear until months later.
 4. This creates two problems:
 - i. The time lag between the cause and the appearance of the problem (the longer the time interval, the more complicated the problem has become)
 - ii. The effect of the system errors on files and records within the system.
1. 识别软件中存在的所有缺陷，消除它们，实现无误运行。
 2. 测试对系统的成功至关重要。
 3. 测试不足或未测试会导致错误，而这些错误可能要几个月后才会出现。
 4. 这带来了两个问题：
 5. 问题发生与出现之间的时间差（时间间隔越长，问题越复杂）
 6. 系统错误对系统内文件和记录的影响。

历史上一些软件故障的例子

Some examples of software failures in history



For over 2 years Nissan recalled over a million cars, thanks to a software glitch in the airbag sensory detectors.



In September 2014, Apple faced an embarrassment after it had to pull out its new iOS software update only after a few hours of its release



In February 2014, Toyota Motor recalled 1.9 million newest-generation Prius vehicles worldwide due to a programming error that caused the car's gas-electric hybrid systems to shut down.



Big Billion Day Sale Flipkart 2014.

In 1996, a software bug in the bank software application caused the bank accounts of 823 customers of a major US bank to be credited with 920 million US dollars.

Software failures in the US cost the economy \$1.1 trillion in assets in 2016. What's more, they impacted 4.4 billion customers.

Week 4: Test Management & Agile Testing Practices **第4周：测试管理与敏捷测试实践**

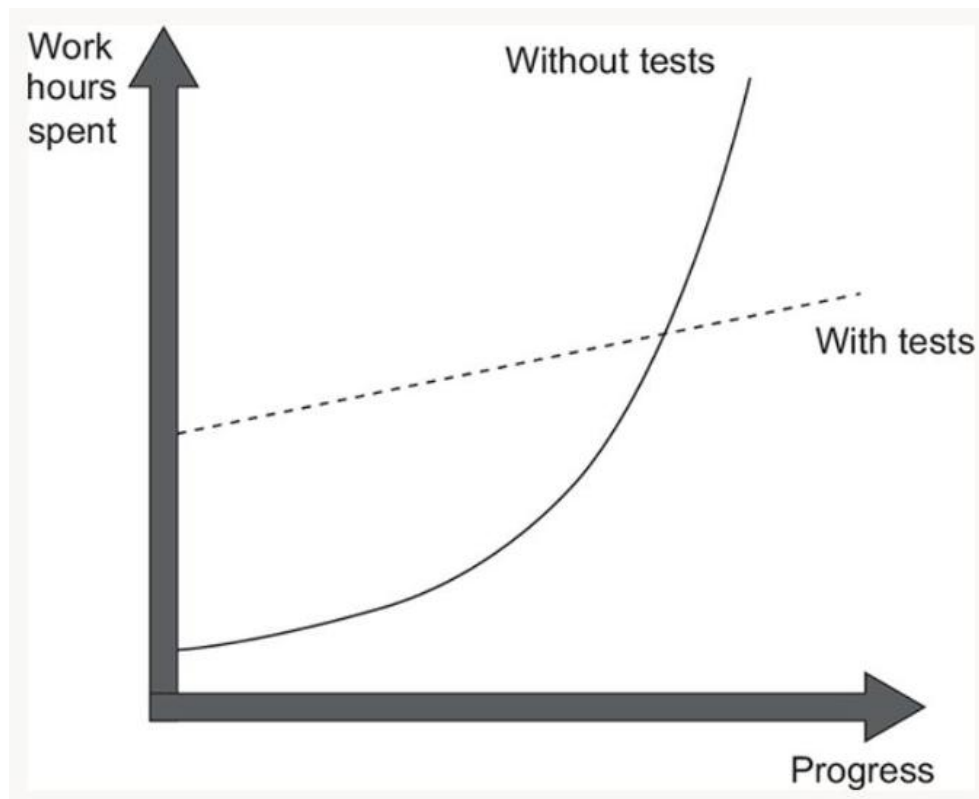
系统测试的目标

Objective of System Testing

- I. Small system error can explode into a much larger problem.
 - II. Effective testing early in the process translates directly into long-term cost savings from a reduced number of errors.
 - III. Important for ensuring customer satisfaction and reliability with the product in the long term.
 - IV. To point out defects and errors made during the software implementation or development phase.
 - V. Better business optimization (low optimization cost).
 - VI. Helps in improving the overall quality of the system.
- I. 小的系统错误可能爆发成更大的问题。
 - II. 在流程早期进行有效的测试，直接转化为长期的成本节约，同时减少错误。
 - III. 这对于确保客户满意度和产品长期可靠性非常重要。
 - IV. 指出软件实施或开发阶段的缺陷和错误。
 - V. 更好的业务优化（低优化成本）。
 - VI. 有助于提升系统的整体质量。

系统测试的目标

Objective of System Testing



The difference in growth dynamics between projects with and without tests. A project without tests has a head start but quickly slows down to the point that it's hard to make any progress.

Week 4: Test Management & Agile Testing Practices **第4周：测试管理与敏捷测试实践**

系统测试的目标

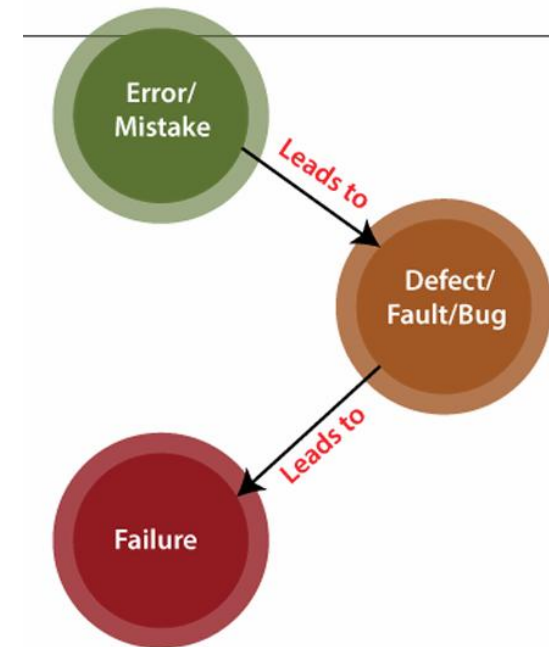
Objective of System Testing

The bugs introduced by programmer inside the code is called as Defect. Defect is defined as the deviation from the actual and expected result of application or software. Developer finds the defect and corrects it by himself during the development phase.

If testers find any mismatch in the application/system in testing phase then they call it as Bug.

A fault is an incorrect step, process or data definition in a software product.

When a defect reaches the end customer, it is called as Failure.



规划你的软件测试

Planning your Software Testing

1. The purpose of a written test plan is to improve the communication among all the people involved in testing application software.
2. Help people outside the test team such as developers, business managers, customers understand the details of testing.
3. The plan specifies each person's role during testing.
4. Also serves as a checklist, whether or not the master test plan has been completed or not. Test Plan guides our thinking.
5. Many times, a master test plan is not just a single document, but a collection of documents.
6. A master test plan is a project within the overall system development project.
7. The Overall Plan and Testing Requirements sections are like a baseline project plan for testing, with a schedule of events, resource requirements, and standards of practice outlined.
8. Testing managers are responsible for creating testing plans, establishing testing standards, integrating testing, and ensuring that test plans are completed.

软件熵

Software Entropy

1. In software, entropy manifests in the form of code that tends to deteriorate.
2. Each time you change something in a code base, the amount of disorder in it, or entropy, increases.
3. If left without proper care, such as constant cleaning and refactoring, the system becomes increasingly complex and disorganized.
4. Fixing one bug introduces more bugs, and modifying one part of the software breaks several others – it's like domino effect.
5. Eventually, the code base becomes unreliable.
6. Worst of all, it is hard to bring it back to stability.
7. Tests help overturn this tendency. They act as a safety net – a tool that provides insurance against a vast majority of regressions.
8. Tests help make sure the existing functionality works, even after you introduce new features or refactor the code to better fit the new requirements.

Week 4: Test Management & Agile Testing Practices **第4周：测试管理与敏捷测试实践**

软件熵

Software Entropy

Example 1: The Discount Function

Imagine an e-commerce system with a simple function that calculates the final price after a discount:

```
def apply_discount(price, discount_percent):  
    if discount_percent < 0 or discount_percent > 100:  
        raise ValueError("Invalid discount")  
    return price * (1 - discount_percent / 100)
```

This works fine. Other parts of the system rely on it: the shopping cart, invoices, reports, etc.

The entropy starts with a “small” change

- ❑ A new requirement arrives: VIP customers get an extra 5% discount on top of the normal discount.

软件熵

Software Entropy

A developer quickly modifies the function:

```
def apply_discount(price, discount_percent, is_vip=False):  
    if discount_percent < 0 or discount_percent > 100:  
        raise ValueError("Invalid discount")  
    vip_multiplier = 0.95 if is_vip else 1.0  
    return price * (1 - discount_percent / 100) * vip_multiplier
```

Looks harmless, but now the return value is always multiplied by vip_multiplier. The function signature changed, and the calculation changed for everyone.

软件熵

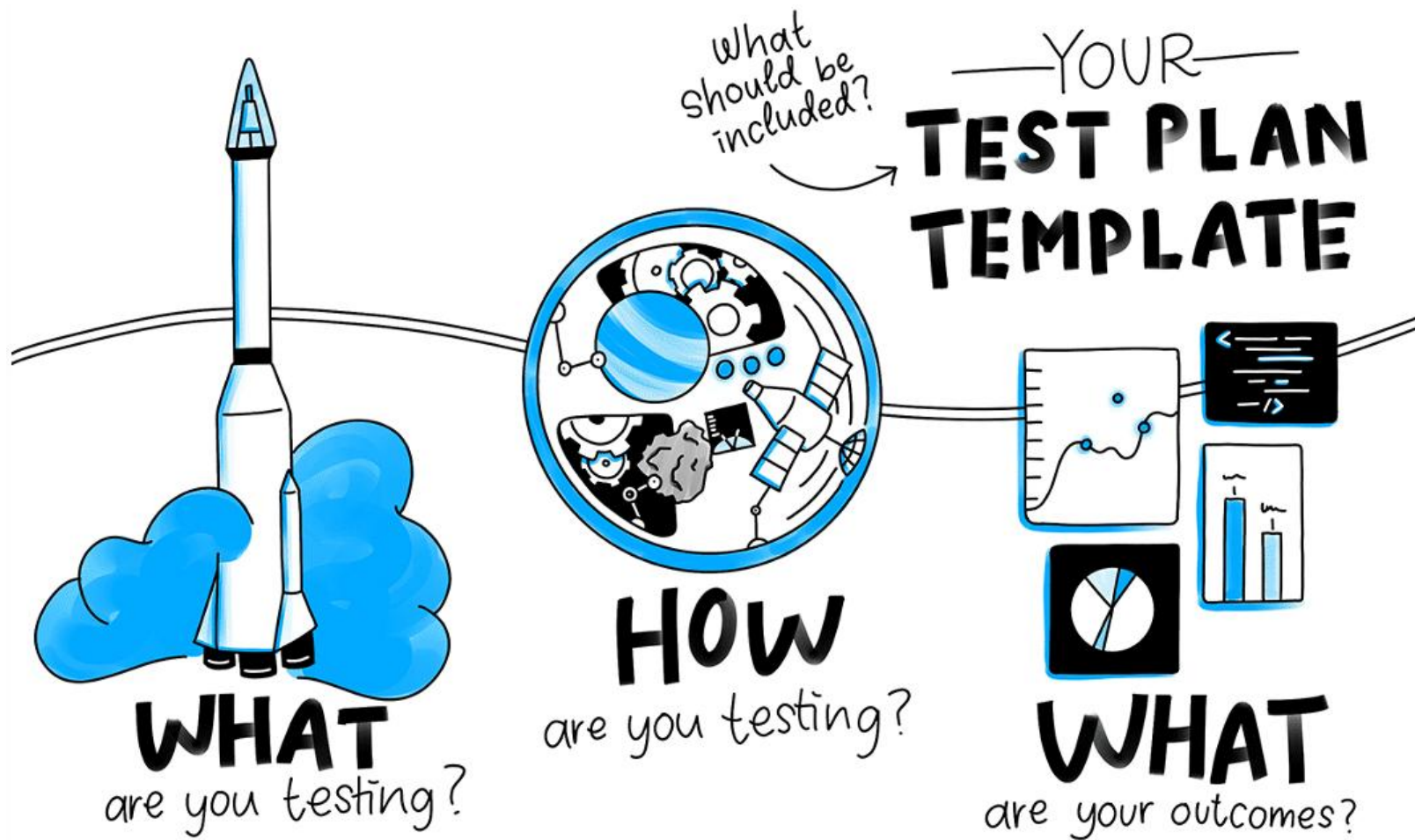
Software Entropy

Domino effect (no tests)

- ❑ The invoice module still calls `apply_discount(price, 20)` without the `is_vip` argument. That's fine—it defaults to `False`—so invoices still work.
- ❑ The reporting module, however, had a cached list of discount percentages and expected the original formula. Now margins are slightly off.
- ❑ A third module, written by another team, monkey-patched a wrapper around `apply_discount` to log discounts—it's now broken because it expected only two arguments.
- ❑ One change, meant only for VIPs, silently rippled through the system. You fix the reporting bug by adding a new parameter, but that breaks the wrapper differently. The code base becomes more tangled with every fix—entropy grows.

简单来说，考试规划就是。

Test planning in simple words.



Week 4: Test Management & Agile Testing Practices 第4周：测试管理与敏捷测试实践

简单来说，考试规划就是。

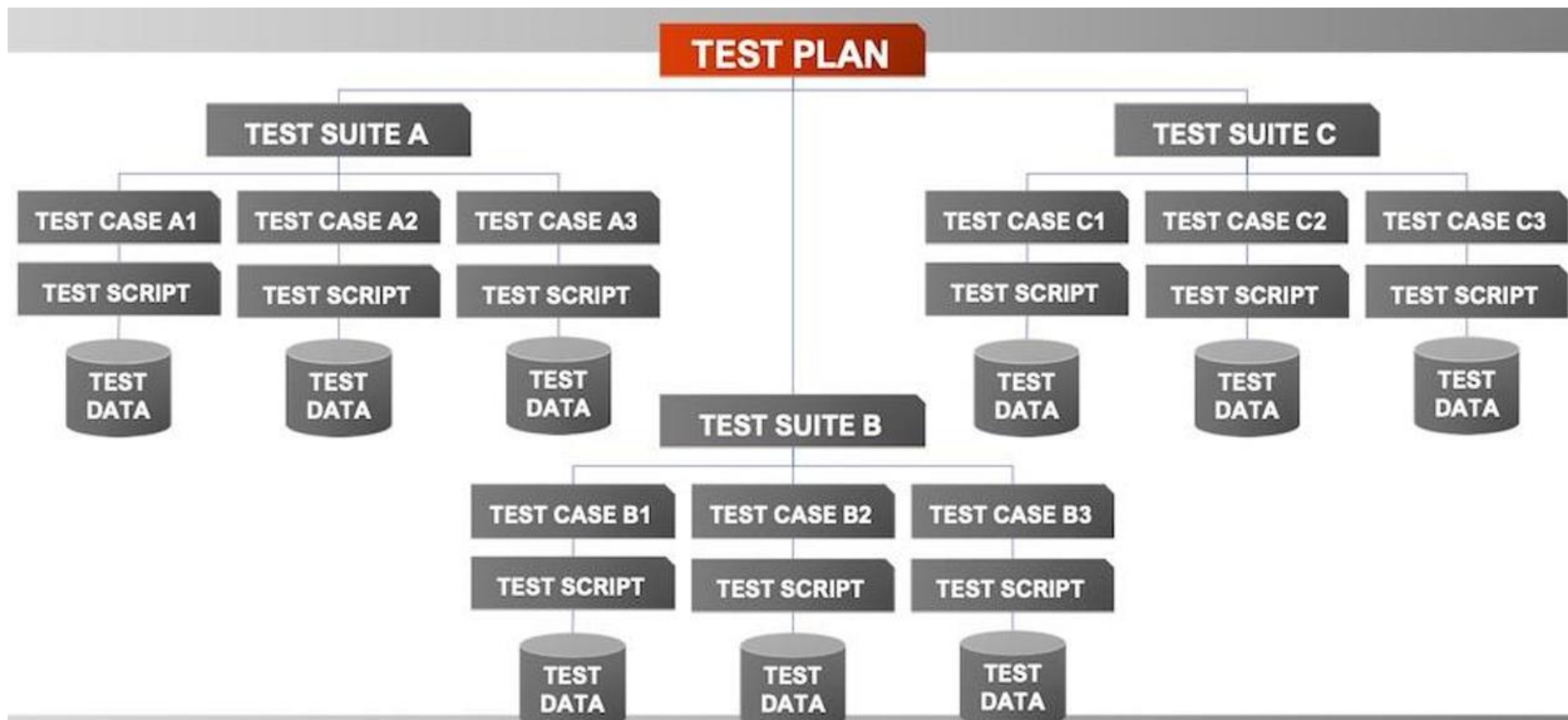
Test planning in simple words.



Week 4: Test Management & Agile Testing Practices **第4周：测试管理与敏捷测试实践**

测试计划

Test Plan



测试计划

Test Plan

1. Software application testing is an umbrella term that covers several types of tests.
2. Mosely (1993) organizes the type of tests according to whether they employ static or dynamic techniques and whether the test is automated or manual.
3. Static Testing means that the code being tested is not executed. The results of running the code are not an issue for that particular test.
4. Dynamic Testing, involves execution of code.
5. Automated testing means the computer conducts the test while Manual Testing means that people do.

	Manual	Automated
Static	Inspections	Syntax checking
Dynamic	Walkthrough	Unit test
	Desk checking	Integration test
		System test

检查、参观

Inspections, Walkthroughs

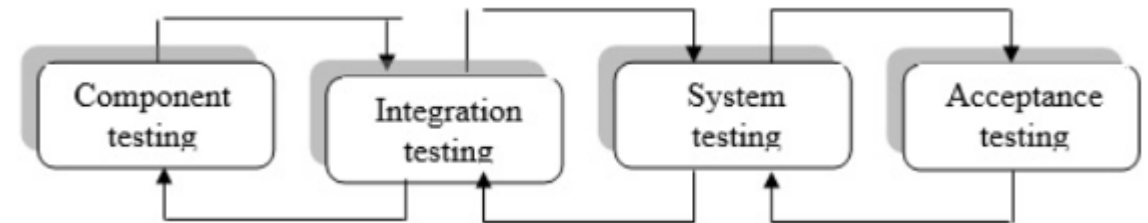
1. Formal group of activities where participants manually examine the code for occurrences of well-known errors.
2. Syntax, grammar, and some other routine errors can be checked by automated inspection software.
3. Walkthrough, in a structured manner, is a very effective method of detecting errors in code.
4. Code walkthroughs should be done frequently when the pieces of work reviewed are relatively small and before the work is formally tested.

单元测试

Unit Testing

1. Sometimes called **module testing**, often done automatically.
2. In unit testing, each module is tested alone in an attempt to discover any error that may exist in the module's code.
3. Except for small programs, system should not tested as a single, monolithic unit.
4. Large programs consist of many smaller pieces – classes, methods, packages, etc.
5. “Unit” is a generic name for these smaller pieces.
6. Three important types of testing are:
 1. Unit Testing (testing units in isolation)
 2. Integration Testing (test integrated units)
 3. System Testing (test entire system that is fully integrated)
7. VII. Unit testing is done to test the smaller pieces in isolation before they are combined with other pieces.

Usually done by developers who write the code.



Week 4: Test Management & Agile Testing Practices 第4周：测试管理与敏捷测试实践

系统测试的目标

Objective of System Testing

1. For the past two decades there has been a push towards adopting unit testing.
2. Unit testing is now considered mandatory in most companies.
3. Unless you are working on a throwaway project, unit testing is necessary.
4. The discussion has shifted from “Should we write unit tests?” to “What does it mean to write good unit tests?”

This is where the main confusion still lies.

5. Results of this confusion are visible in software projects.
 - i. Many projects have automated tests; they may even have lots of them.
 - ii. But existence of those tests often doesn't provide the results the developers hope for.
 - iii. Unit tests that are supposed to help do not seem to help when new features take forever to implement, new bugs constantly appear in the already implemented and accepted functionality.

单元测试与代码设计的关系

Relationship between unit testing and code design

1. If you find a code is hard to unit test, it is a strong sign that the code needs improvement.
2. The poor quality usually manifests itself in tight coupling, which means different pieces of production code are not decoupled from each other enough, and it is hard to test them separately.
3. But, if you code is easy to unit test does not necessarily indicate its good quality.
4. Thus, the goal of unit test is to enable sustainable growth of the software project
5. It is quite easy to grow a project, especially when you start from scratch.
6. It's much harder to sustain this growth over time.

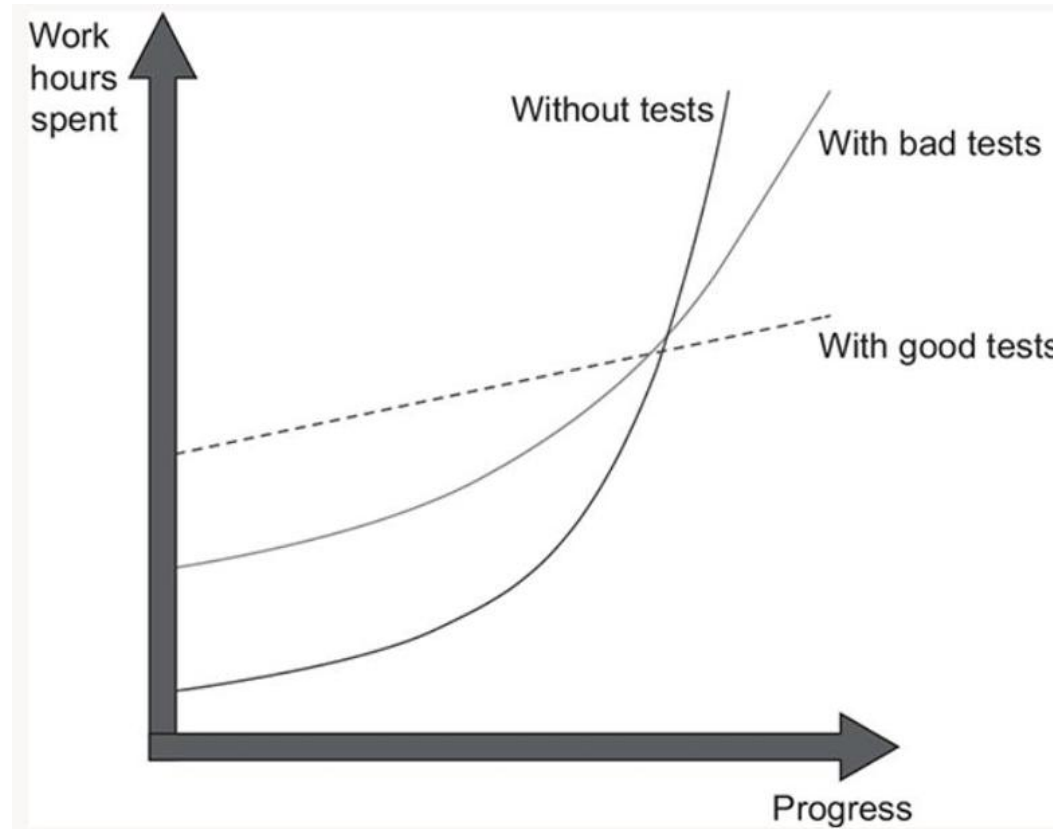
什么样的测试是好或坏的?

What makes a good or bad test?

- ❑ It is not enough to just write tests.
- ❑ Badly written tests still result in the same picture.
- ❑ Bad tests do help to slow down code deterioration at the beginning. But nothing really changes in the grand scheme of things.
- ❑ It might take longer for such a project to enter the stagnation phase, but stagnation is still inevitable.
- ❑ Not all tests are created equal.
- ❑ Bad tests can raise false alarms, don't help you catch regression errors, and are slow and difficult to maintain.
- ❑ It's easy to fall into the trap of writing unit tests for the sake of unit testing without a clear picture of whether it helps the project.
- ❑ Things to remember : test's value and its upkeep cost.

什么样的测试是好或坏的？

What makes a good or bad test?



The difference in growth dynamics between projects with good and bad tests. A project with badly written tests exhibits the properties of a project with good tests at the beginning, but it eventually falls into the stagnation phase.

Week 4: Test Management & Agile Testing Practices **第4周：测试管理与敏捷测试实践**

集成测试

Integration Testing

1. Have you ever been in a situation where all the unit tests pass but the application still doesn't work?
2. You can never be sure your system works as a whole if you rely on unit tests exclusively.
3. Validating software components in isolation from each other is important, but it's equally important to check how those components work in integration with external systems.
4. This is where integration testing comes into play.
5. You have to validate how different parts of a system integrate with each other and external systems: the database, the message bus, and so on.

集成测试

Integration Testing

1. Integration testing is done by combining the modules and testing them together.
 2. It is gradual.
 3. The integration plan is guided by the module dependency graph of the structure chart.
 4. Modules are typically integrated in a top-down, incremental order.
 5. You continue this procedure until the entire program has been tested as a unit.
1. 集成测试是通过组合模块并共同测试来完成的。
 2. 这是渐进的。
 3. 整合计划由结构图的模块依赖图指导。
 4. 模块通常以自上而下、增量顺序集成。
 5. 你会一直这样做，直到整个项目作为一个整体进行测试。

车手与小作品

Drivers and Stubs

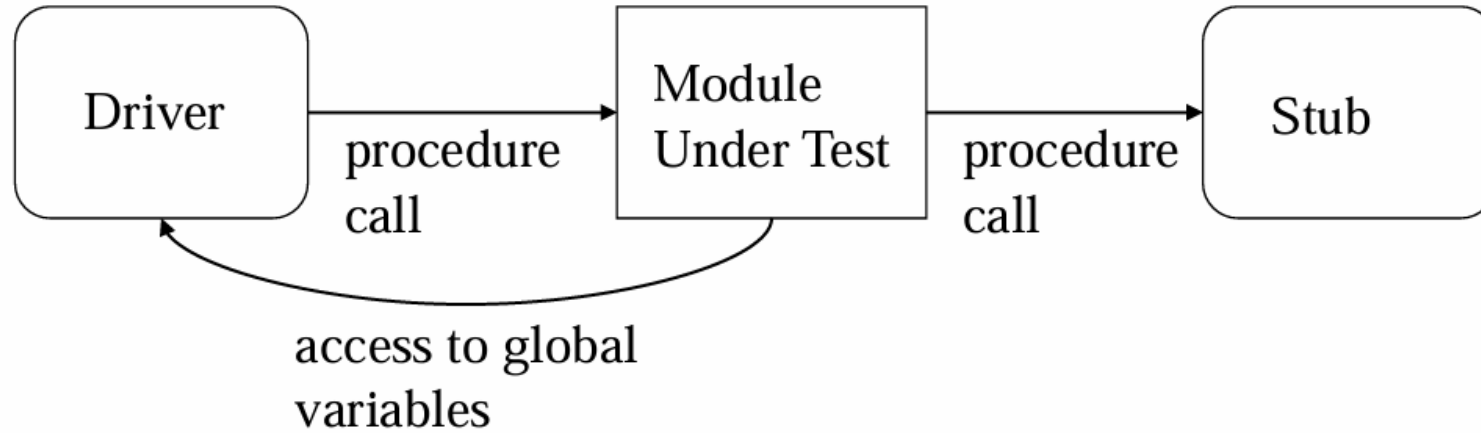
- ❑ It is almost always a good idea to develop and test software in “pieces”.
- ❑ At first look this may seem impossible because it is hard to imagine how one can test one “piece” if the other “pieces” that it uses have not yet been developed.
- ❑ The way around this is to use stubs and drivers.

Simply:

- ❑ **STUB** A piece of code that simulates the activity of missing components.
- ❑ **DRIVER** A piece of code that passes test cases to another piece of code.

车手与小作品

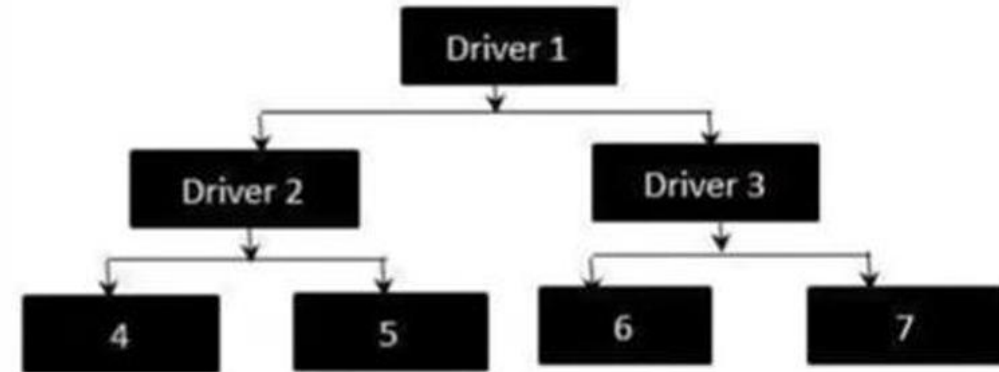
Drivers and Stubs



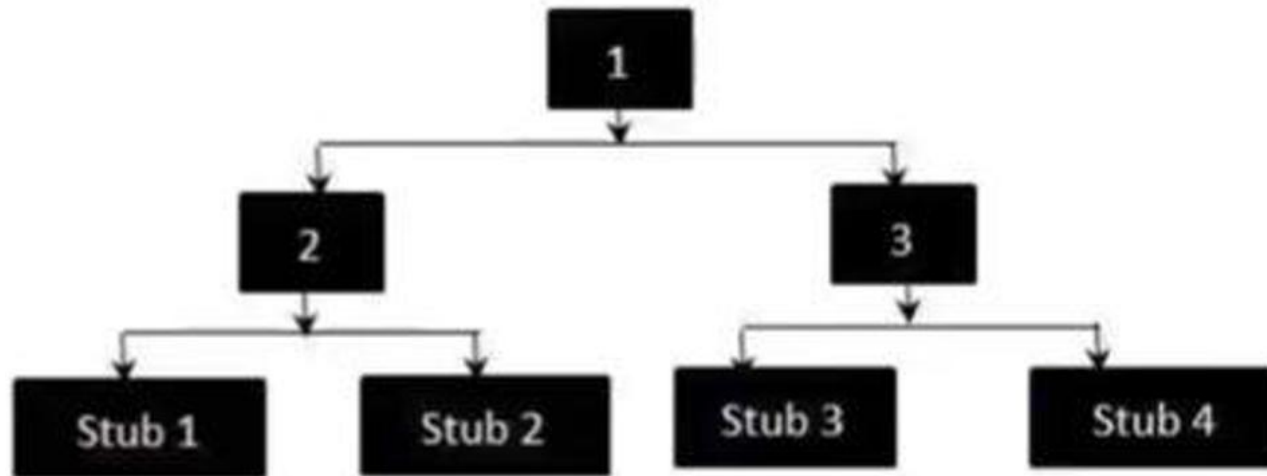
- ❑ Driver and Stub should have the same interface as the modules they replace
- ❑ Driver and Stub should be simpler than the modules they replace

验证与确认技术

Objective of System Testing



[Video](#)



Week 4: Test Management & Agile Testing Practices **第4周：测试管理与敏捷测试实践**

示例

Example

- We need to develop a “calculator” that converts between different units (e.g. feet to inches)
- We “stub out” the UnitConverter class.
- This “stubbed-out” class contains comments that describe the class and all of the methods that we expect it to have.

In summary:

- **Stubs**

are created during top-down integration. i.e Allow the testing of the upper level module when the lower level module is not yet developed.

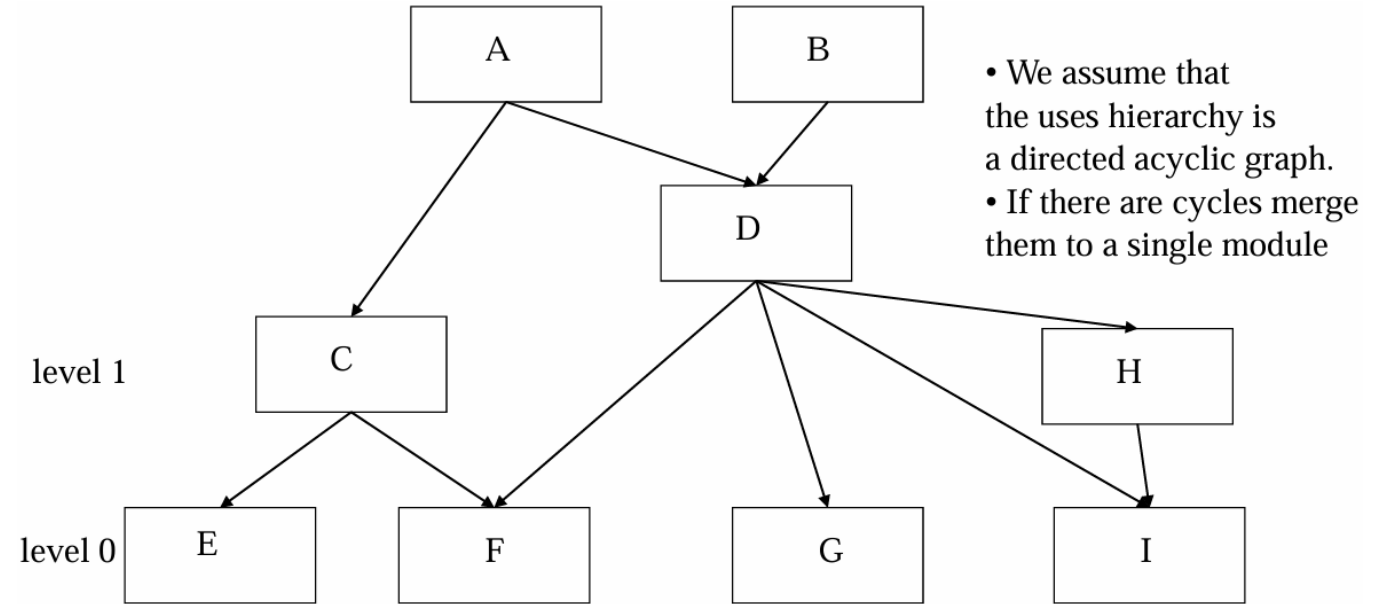
- **Drivers**

are created during bottom-top integration. i.e. Allow testing of the lower level modules when upper level modules are not yet developed.

模块结构

Module Structure

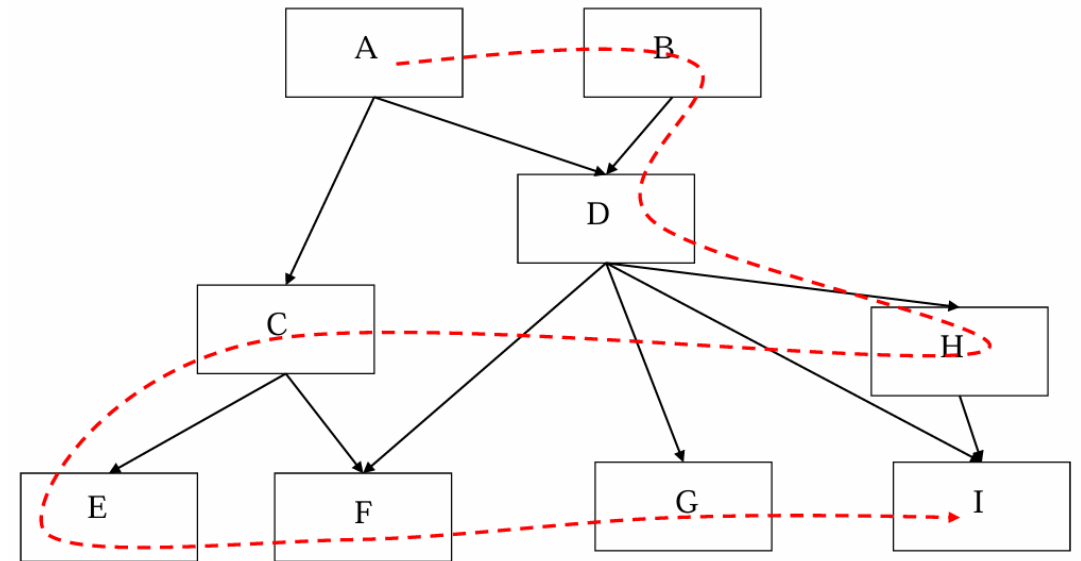
- ❑ A uses C and D; B uses D; C uses E and F; D uses F, G, H and I; H uses I
- ❑ Modules A and B are at level 3; Module D is at level 2; Modules C and H are at level 1; Modules E, F, G, I are at level 0
- ❑ level 0 components do not use any other components
- ❑ level i components use at least one component on level i-1 and no component at a level higher than i-1



自上而下集成方法

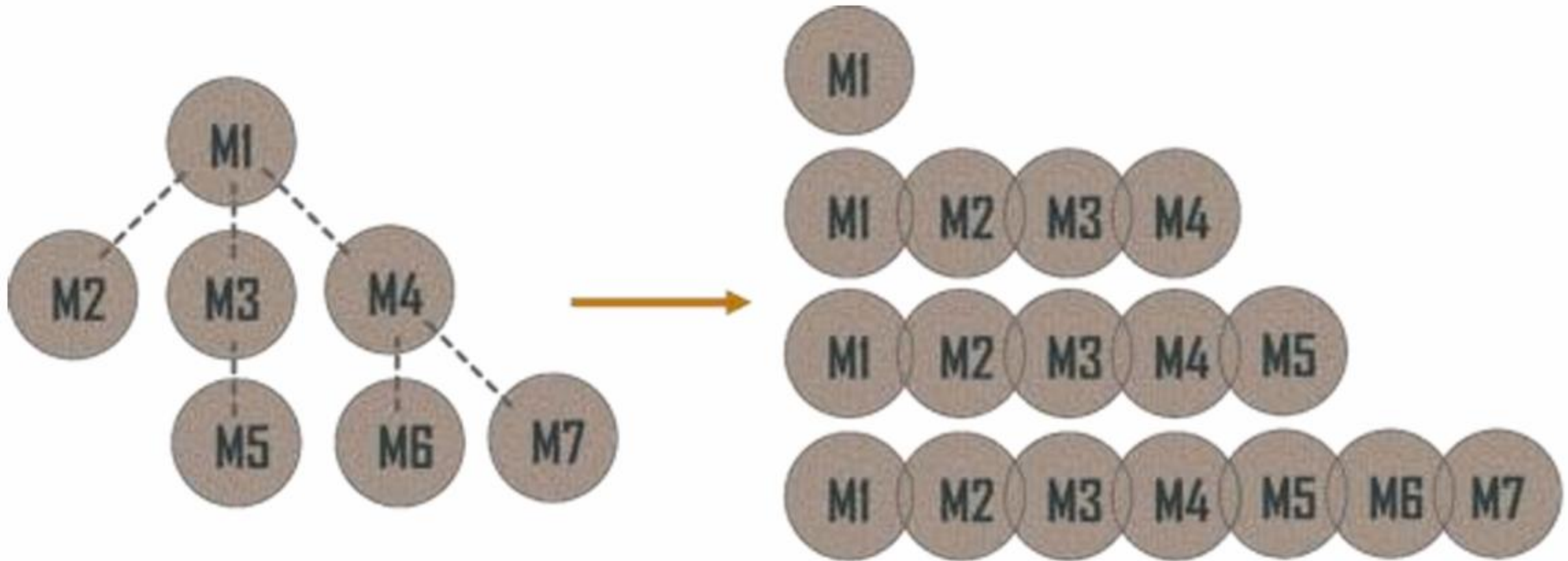
Top-down integration approach

1. Integration starts with the root module and one or two sub modules followed by testing.
2. After the top-level modules are tested, the immediate module is combined and become ready for testing.
3. It is suitable for small systems.
4. Advantage : Depth-first strategy make the parts of system available for early demo.
5. Building stubs adequate to test upper levels may require a lot of effort.
6. A lot of effort (later discarded) is put into designing and implementing drivers and stubs.



自上而下集成方法

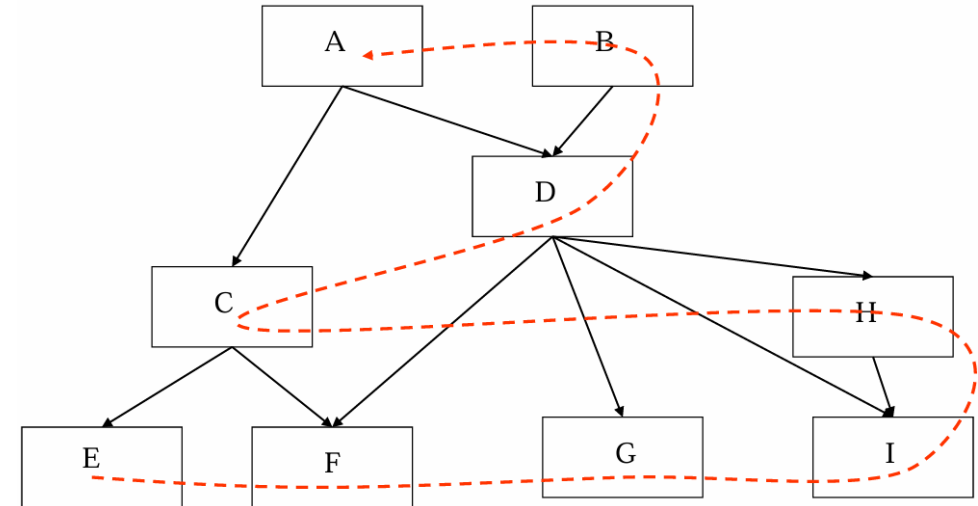
Top-down integration approach



上下积分

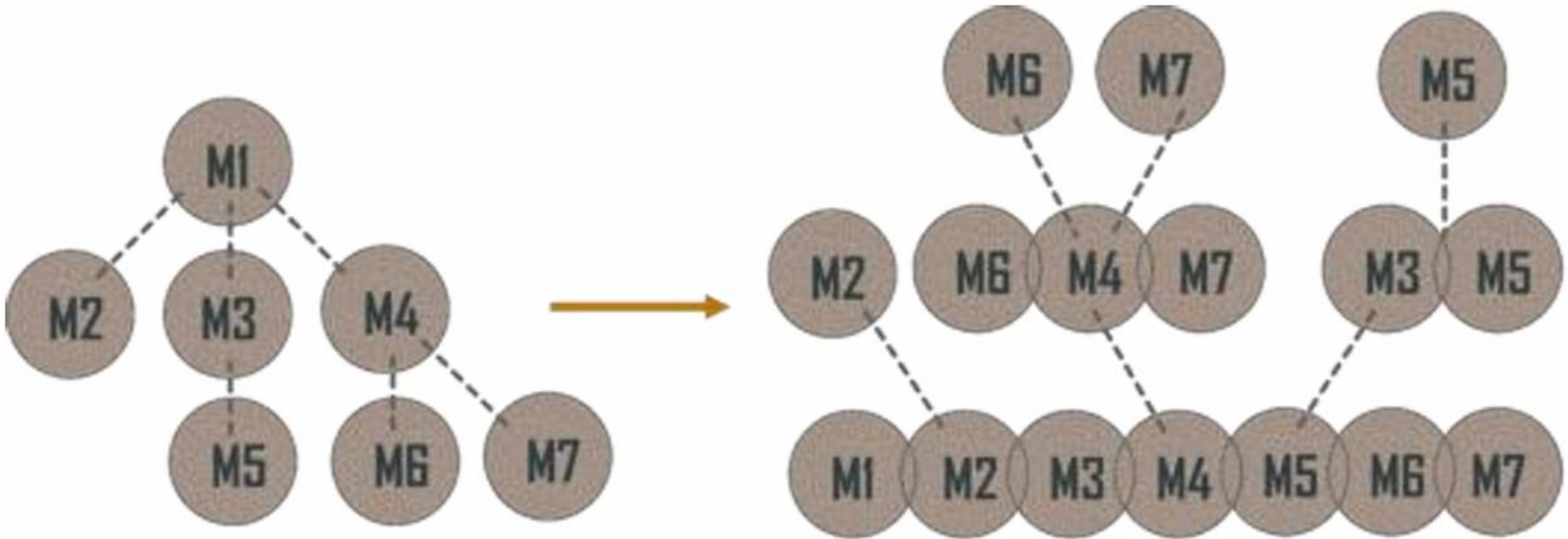
Bottom-top integration

- ❑ Up - Start with atomic (leaf) modules.
- ❑ Use drivers to tests clusters of modules.
- ❑ Gradually add calling modules and move testing upward.
- ❑ The purpose of testing each sub module is to test the interfaces among them.
- ❑ In this situation control and data interfaces between modules are tested.
- ❑ The advantage is, many sub systems, having no dependency, can be tested simultaneously.
- ❑ The disadvantage is that during testing a number of sub systems makes the situation complex.
- ❑ This extreme condition calls for big bang approach



上下积分

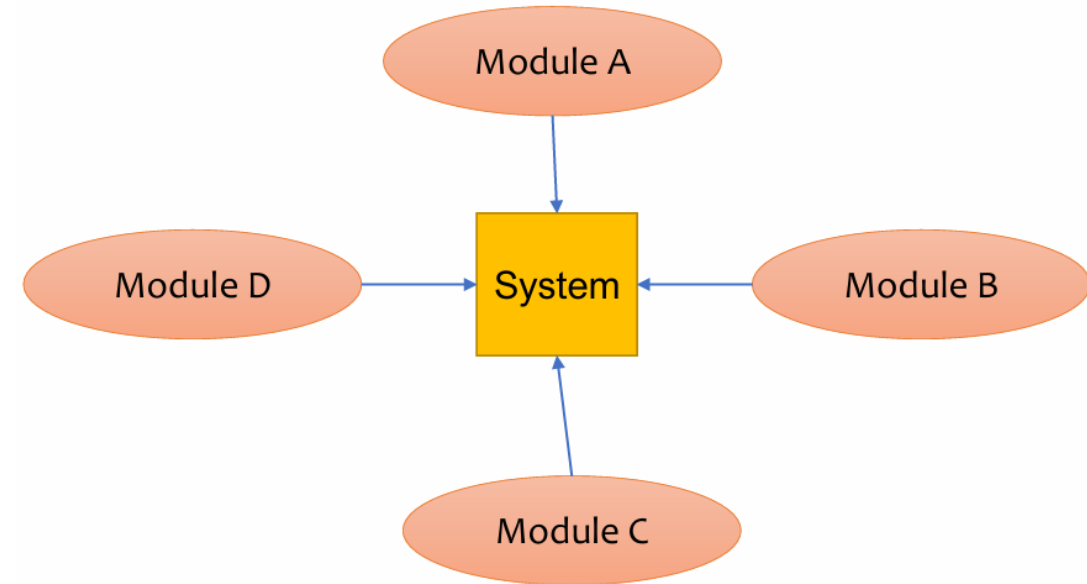
Bottom-top integration



大爆炸方法

Big Bang Approach

- ❑ All the sub modules are integrated in a single step and tested together.
- ❑ A form of non-incremental integration testing.
- ❑ This approach is convenient for a small system where on-go-the errors are fixed.
- ❑ Fault Localization Issues: The disadvantage is to spot the error found in the exact module during integration testing.
- ❑ To remove the error during integration testing is costly and time consuming.
- ❑ Generally followed by those developers who follow “Run and See” approach.
- ❑ Chances of having critical failures are more because of integrating all the components together at the same time.



混合集成方法

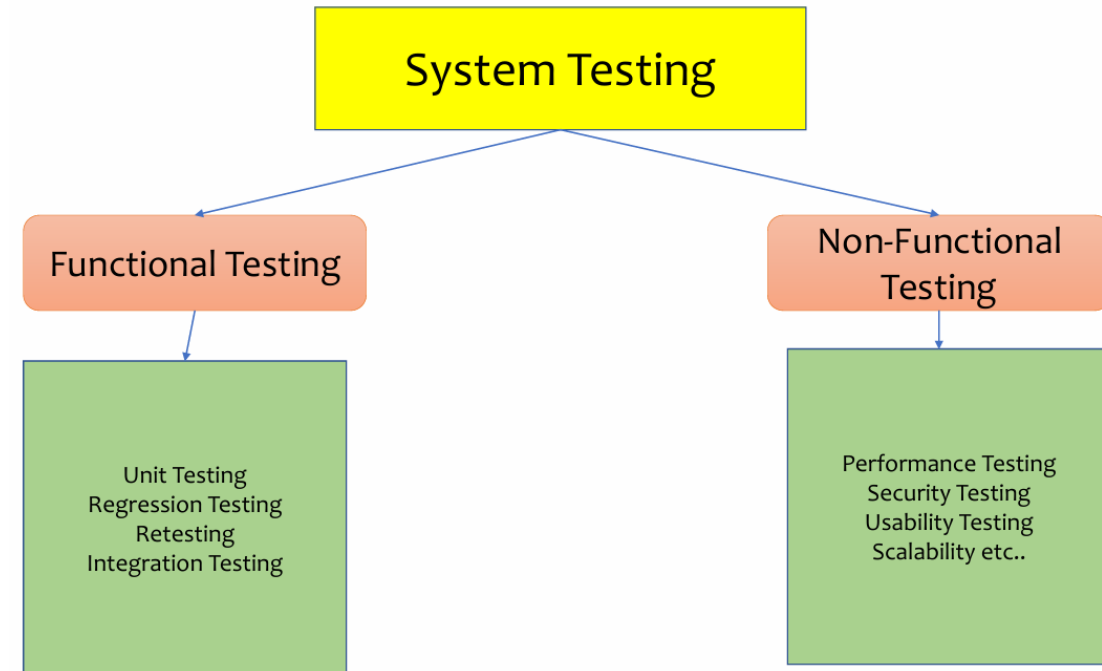
Mixed Integration Approach

1. Is a combination of both top-down and bottom-up approaches.
2. Also known as **sandwiched integration, hybrid integration testing**.
3. The disadvantages of both the approaches are reduced to a great extent.
4. The testing is possible from both top and bottom level, wherever the modules are available.
5. This is common approach for integration testing .
6. In top-down integration, integration testing can start only once the top-level modules are coded and unit tested while in bottom-up approach integration testing can be started only when bottom level modules are coded and unit tested.
7. Mixed integration testing approach overcomes those limitations by making sure integration testing of the modules can be started as and when they are available after unit testing.
8. Top-level modules are tested with lower level modules and lower level modules are tested with high-level modules simultaneously.
9. Mixed integration testing requires to create both the stubs and the drivers.

系统测试

System Testing

- ❑ Is similar to integration testing, but instead of integrating modules into programs for testing, you integrate programs into systems.
- ❑ Tests are performed to know if a complete build aligns with the functional and non-functional requirements made for it.
- ❑ System testing follows the same incremental logic that does the integration testing.
- ❑ Programs are typically integrated in top-down incremental fashion.
- ❑ Under both integration and system testing, the individual modules and programs get tested many times as well as interfaces between modules and programs also get tested.



系统测试通常在什么时候进行？

When is system testing done?

- ☐ During the development of each software version.
- ☐ During the application launch.
- ☐ When unit and integration testing are complete.
- ☐ This test level checks the software's conformity to user demands, functional performance, and design.
- ☐ 在每个软件版本的开发过程中。
- ☐ 在应用启动期间。
- ☐ 当单元测试和集成测试完成时。
- ☐ 该测试层级检查软件是否符合用户需求、功能性能和设计。

系统测试的目标

Objective of System Testing

Alpha testing and Beta Testing

❑ Alpha Testing

- ❑ Tests performed at the developer's site before the system is finally installed in the real working environment (user's site).
- ❑ Involves testing the system with live data supplied by the organization rather than by the test data used by the system designer.

❑ Beta Testing

- ❑ The system is delivered to a number of potential users who agree to use that system and provide feedback to the designers.

Alpha测试和Beta测试

Alpha测试

- ❑ 在系统最终安装到真实工作环境（用户现场）之前，在开发者现场进行的测试。
- ❑ 涉及使用组织提供的实时数据测试系统，而非系统设计者使用的测试数据。

测试版

- ❑ 该系统会交付给一些潜在用户，他们同意使用该系统并向设计者提供反馈。

验收测试

Acceptance Testing

- ❑ The final stage in the testing process before the system is accepted for operational use by the client.
- ❑ System is tested with data supplied by the user than with simulated data.
- ❑ May reveal errors and omissions in the system requirement definition.
- ❑ May also reveal requirements problems where the system's facilities do not really meet the user's needs or the system performance is not acceptable.
- ❑ 这是系统被客户接受运营前的最后测试阶段。
- ❑ 系统测试时使用用户提供的数据,而非模拟数据。
- ❑ 可能会发现系统需求定义中的错误和遗漏。
- ❑ 也可能揭示当系统功能未能真正满足用户需求或系统性能不合格时的需求问题。

测试策略

The Test Strategy

1. Prepare test plan.

Outputs expected from the system. Criteria for evaluating outputs. A volume of test data. Procedure for using test data. Personnel and training requirements.

2. Specify conditions for user acceptance testing.

Analyst and the user should agree on the conditions for the test.

3. Prepare test data for program testing.

Test data should be prepared and documented to ensure that all aspects of the program are properly tested.

4. Prepare test data for transaction path testing.

Develops the data required for testing every condition and transaction to be introduced into the system. The path of each transaction from origin to destination is carefully tested for reliable results.

5. Plan user training.

6. Compile/assemble programs.

7. Prepare job performance aids.

Materials to be used by personnel to run the system are specified and scheduled. Eg. Posted instruction schedule to load the data/disk drive or to a help menu.

8. Prepare operational documents.

Mainly related to setting some performance criteria to be used.

Turn around time, back up, file protection, human factor.

验证与确认技术

Guidelines for module testing

- ❑ To emphasize the importance and applicability to module testing the following guidelines may be followed:
- ❑ Try to design module tests so that failure to pass one test case will not move to the next test case.
- ❑ Under stressful load and conditions, test the modules at their performance limit and beyond.
- ❑ Measure the performance characteristics like throughput accuracy, input and output capacities, and timings should be measured for each module.
- ❑ Concurrently many problems should not be solved. They should be handled one after the other so that the source of error can be traced easily.
- ❑ Test data as input should be valid or representative of the data to be provided by the user. If not, then the reliability of the output is suspected.
- ❑ If using artificial data, make sure all combinations of values and formats are input to the system.

Week 4: Test Management & Agile Testing Practices **第4周：测试管理与敏捷测试实践**

测试框架

Testing Frameworks

Following are the different testing frameworks:

- ❑ **jUnit-** for Java unit test.
- ❑ **Selenium-** is a suite of tools for automating web applications for software testing purposes, plugin for Firefox.
- ❑ **HP-QC-** is the HP Web-based test management tool. It familiarizes with the process of defining releases, specifying requirements, planning tests, executing tests, tracking defects, alerting on changes, and analyzing results. It also shows how to customize project.
- ❑ **IBM Rational-** Rational software has a solution to support business sector for designing, implementing and testing software.

验证与确认技术

Conclusion

- ❑ Software Testing is done to ensure that it runs correctly, including hardware and other software linked to it.
- ❑ Planning, discipline, procedure control and documentations are vital for successful software testing.
- ❑ Planning for the test phase should begin early in the development life cycle and should be constant concern throughout the development cycle.
- ❑ 软件测试旨在确保其正常运行，包括与之关联的硬件及其他软件。
- ❑ 规划、纪律、程序控制和文档对于成功的软件测试至关重要。
- ❑ 测试阶段的规划应从开发生命周期早期开始，并在整个开发周期中持续关注。